Estudo Dirigido: Simulação Estatística em R e Python

Implementação e Análise de Conceitos Fundamentais

29 de setembro de 2025

Resumo

Este documento explora a interpretação de conceitos estatísticos fundamentais (Confiança do Intervalo, Estimadores Não-Viciados e Erro Tipo I) através de simulações em **R** e **Python**. O objetivo é demonstrar numericamente o significado de cada conceito por meio da repetição de processos de amostragem.

1 Simulação do Nível de Confiança (Intervalo de Confiança - IC)

1.1 Teoria

O **Nível de Confiança** (ex: 95%) **não** é a probabilidade de que um IC específico contenha a média populacional (μ). É um **exercício de imaginação**: se você coletasse infinitas amostras nas mesmas condições e calculasse um IC para cada uma, a confiança seria a **proporção de intervalos que contêm a verdadeira média populacional**.

- **População Gerada:** $N = 100.000$, $\mu = 180$, $\sigma = 20$
- **Amostra:** $n = 100$
- **Confiança Fixada:** 95%

1.2 Código em R

O código a seguir gera a população, coleta 10.000 amostras e verifica quantos ICs de 95% contêm $\mu = 180$.

Listing 1: Simulação do IC em R

```r
# 1. Gerar a populacao
pop = rnorm(100000, 180, 20)
mu = 180

# 2. Inicializar vetores
x = NULL
l = NULL # Vetor logico para armazenar se o IC contem mu

# 3. Loop de 10.000 simulacoes
for(i in 1:10000){
  x = sample(pop, 100)

  # Calcula o IC e verifica se mu (180) esta contido
  # t.test(x)$conf.int[1] < 180 & t.test(x)$conf.int[2] > 180
  intervalo = t.test(x)$conf.int
  l[i] = (intervalo[1] < mu & intervalo[2] > mu)
}

# 4. Resultado: Proporcao de ICs que contem mu
# sum(l) soma os valores TRUE (1), length(l) e o total (10000)
proporcao_sucesso = sum(l) / length(l)
print(proporcao_sucesso)
```

1.3 Código em Python

O código Python equivalente utiliza a biblioteca `scipy` para o cálculo do Intervalo de Confiança.

Listing 2: Simulação do IC em Python

```python
import numpy as np
from scipy.stats import t

# 1. Parametros e Geracao da Populacao
POP_SIZE = 100000
TRUE_MEAN = 180
TRUE_STD = 20
CONFIDENCE_LEVEL = 0.95
SAMPLE_SIZE = 100
NUM_SIMULATIONS = 10000

np.random.seed(42)
population = np.random.normal(TRUE_MEAN, TRUE_STD, POP_SIZE)
ic_contains_mean = []

# 2. Loop de Simulacao
for i in range(NUM_SIMULATIONS):
    sample = np.random.choice(population, SAMPLE_SIZE, replace=False)

    # Calculo do IC
    sample_mean = np.mean(sample)
    sample_std = np.std(sample, ddof=1) # ddof=1 para estimador nao viciado (n-1)
    standard_error = sample_std / np.sqrt(SAMPLE_SIZE)
    degrees_freedom = SAMPLE_SIZE - 1

    # Usa t.interval da scipy para calcular o IC
    ic_lower, ic_upper = t.interval(
        confidence=CONFIDENCE_LEVEL,
        df=degrees_freedom,
        loc=sample_mean,
        scale=standard_error
    )

    # 3. Verifica se o IC contem a media populacional
    contains_mean = (ic_lower <= TRUE_MEAN) and (TRUE_MEAN <= ic_upper)
    ic_contains_mean.append(contains_mean)

# 4. Resultado Final
intervals_containing_mean = np.sum(ic_contains_mean)
proportion_success = intervals_containing_mean / NUM_SIMULATIONS

print(f"ICs que contiveram \mu: {intervals_containing_mean}")
print(f"Proporcao de Sucesso: {proportion_success:.4f}")
# Espera-se um valor proximo de 0.95
```

2 Estimadores Não-Viciados e Teorema Central do Limite (TCL)

2.1 Teoria

Um **Estimador Não-Viciado** é aquele cuja **Esperança** é igual ao parâmetro populacional.

- **Média Amostral ($\bar{X}$):** Estimador não-viciado para μ. $E(\bar{X}) = \mu$.
- **Desvio Padrão Viciado (se):** Utiliza denominador n: $\mathbf{se} = \sqrt{\frac{\sum(x_i - \bar{x})^2}{n}}$. Tende a subestimar σ.
- **Desvio Padrão Não-Viciado (s):** Utiliza denominador $n-1$: $\mathbf{s} = \sqrt{\frac{\sum(x_i - \bar{x})^2}{n-1}}$.

2.2 Simulação da Média Amostral ($E(\bar{X})$)

O objetivo é mostrar que a média das médias amostrais converge para $\mu = 180$.

2.2.1 Código em R

Listing 3: Simulacao $E(\bar{X})$ em R

```r
# Usando a mesma populacao do Item 1 (mu=180)
pop = rnorm(100000, 180, 20)
media = NULL # Vetor para armazenar as medias amostrais

for(i in 1:10000){
  x = sample(pop, 100)
  media[i] = mean(x) # Calcula a media de cada amostra
}

# Espera-se que este valor se aproxime de 180
print(mean(media))
```

2.2.2 Código em Python

Listing 4: Simulacao $E(\bar{X})$ em Python

```python
# Usando a mesma populacao do Item 1 (TRUE_MEAN=180)
TRUE_MEAN = 180
SAMPLE_SIZE = 100
NUM_SIMULATIONS = 10000
population = np.random.normal(TRUE_MEAN, 20, 100000)

sample_means = []

for i in range(NUM_SIMULATIONS):
    sample = np.random.choice(population, SAMPLE_SIZE, replace=False)
    sample_means.append(np.mean(sample))

# Media das medias amostrais ($E(\bar{X})$)
mean_of_sample_means = np.mean(sample_means)

print(f"Media das Medias Amostrais (E(\bar{{X}})): {mean_of_sample_means:.4f}")
print(f"Valor Teorico (\mu): {TRUE_MEAN}")
# Espera-se um valor muito proximo de 180.
```

2.3 Simulação do Viés do Estimador de Desvio Padrão (se)

O objetivo é mostrar que o estimador viciado (se) subestima $\sigma = 20$.

2.3.1 Código em R

Listing 5: Simulacao de **se** vs **s** em R

```r
# Definir a funcao para o estimador viciado (denominador n)
se_biased = function(x){
  # Equivalente a np.std(x, ddof=0)
  return(sqrt(sum((x-mean(x))^2)/length(x)))
}

# Usando a mesma populacao (sigma=20)
pop = rnorm(100000, 180, 20)
see = NULL # Vetor para armazenar os desvios padrao viciados

for(i in 1:10000){
```

```
  x = sample(pop, 100)
  see[i] = se_biased(x) # Calcula o se viciado
}

print(paste("Media de se (Viciado):", mean(see)))

# Comparacao com o estimador nao viciado (sd() em R, usa n-1)
s_unbiased = NULL
for(i in 1:10000){
  x = sample(pop, 100)
  s_unbiased[i] = sd(x)
}
print(paste("Media de s (Nao-Viciado):", mean(s_unbiased)))
# Espera-se um valor mais proximo de 20 para s_unbiased
```

2.3.2 Código em Python

Listing 6: Simulacao de **se** vs **s** em Python

```
# 1. Funcao para o estimador viciado (denominador n)
def se_biased(x):
    return np.std(x, ddof=0) # ddof=0 usa denominador n

# 2. Simulacao
TRUE_STD = 20
SAMPLE_SIZE = 100
NUM_SIMULATIONS = 10000
population = np.random.normal(180, TRUE_STD, 100000)

sample_stds_biased = []
sample_stds_unbiased = []

for i in range(NUM_SIMULATIONS):
    sample = np.random.choice(population, SAMPLE_SIZE, replace=False)
    sample_stds_biased.append(se_biased(sample))
    sample_stds_unbiased.append(np.std(sample, ddof=1)) # Denominador n-1

# 3. Resultados
mean_of_biased_stds = np.mean(sample_stds_biased)
mean_of_unbiased_stds = np.mean(sample_stds_unbiased)

print(f"Media do Estimador Viciado (se): {mean_of_biased_stds:.4f}")
print(f"Media do Estimador Nao-Viciado (s): {mean_of_unbiased_stds:.4f}")
print(f"Valor Teorico (\sigma): {TRUE_STD}")
```

3 Comparação de Algoritmos (Teste t Não Pareado)

Esta seção usa dados reais para comparar o tempo de execução de dois algoritmos, A e B.

3.1 Dados

```
Algoritmo A (n=20): 1062, 1081, 1116, 1053, 1103, 1086, 1135, 1133, 1136, 1123, 1095, 1132, 1098
Algoritmo B (n=18): 1073, 1141, 1051, 1040, 1113, 1111, 1099, 1028, 1059, 1081, 1065, 1106, 1019
```

3.2 Teste de Variâncias (Teste F)

3.2.1 Hipóteses

$$\{ H_0 : \sigma_A^2 = \sigma_B^2 H_A : \sigma_A^2 \neq \sigma_B^2$$

3.2.2 Código em R

Listing 7: Teste F-Snedecor (Variancias) em R

```r
a = c(1062, 1081, 1116, 1053, 1103, 1086, 1135, 1133, 1136, 1123, 1095, 1132, 1098,
    1069, 1058, 1092, 1073, 1110, 1104, 1061)
b = c(1073, 1141, 1051, 1040, 1113, 1111, 1099, 1028, 1059, 1081, 1065, 1106, 1019,
    1130, 1120, 1070, 1083, 1010)

# Testa H0: sigma_A^2 = sigma_B^2
var.test(a, b)
```

3.3 Teste t Não Pareado (Teste de Médias)

3.3.1 Hipóteses

$$\{ H_0 : \mu_A = \mu_B H_A : \mu_A \neq \mu_B$$

Assumimos $\sigma_A^2 = \sigma_B^2$ (variancias iguais), baseado em um alto p-valor no teste anterior.

3.3.2 Código em R

Listing 8: Teste t para Médias em R

```r
a = c(1062, 1081, 1116, 1053, 1103, 1086, 1135, 1133, 1136, 1123, 1095, 1132, 1098,
    1069, 1058, 1092, 1073, 1110, 1104, 1061)
b = c(1073, 1141, 1051, 1040, 1113, 1111, 1099, 1028, 1059, 1081, 1065, 1106, 1019,
    1130, 1120, 1070, 1083, 1010)

# Testa H0: mu_A = mu_B, assumindo variancias iguais
t.test(a, b,
       alternative = "two.sided",
       mu = 0, # Diferenca de medias = 0
       paired = FALSE,
       var.equal = TRUE, # Assumindo que as variancias sao iguais
       conf.level = 1 - 0.05)
```

3.3.3 Código em Python

Listing 9: Teste t para Médias em Python

```python
from scipy.stats import ttest_ind

a = np.array([1062, 1081, 1116, 1053, 1103, 1086, 1135, 1133, 1136, 1123, 1095, 1132,
     1098, 1069, 1058, 1092, 1073, 1110, 1104, 1061])
b = np.array([1073, 1141, 1051, 1040, 1113, 1111, 1099, 1028, 1059, 1081, 1065, 1106,
     1019, 1130, 1120, 1070, 1083, 1010])

# Testa H0: mu_A = mu_B
# equal_var=True corresponde a var.equal=TRUE
t_stat, p_value = ttest_ind(a, b, equal_var=True)

print(f"Estatistica t: {t_stat:.4f}")
print(f"P-valor: {p_value:.4f}")
# Se p-valor < 0.05, rejeita-se H0, concluindo que ha diferenca significativa.
```

4 Simulação do Erro Tipo I (α)

4.1 Teoria

O **Erro Tipo I** (α) é a probabilidade de **rejeitar a Hipótese Nula** (H_0) quando ela é, de fato, **verdadeira**. O nível de significância (α) escolhido é a taxa máxima aceitável de Erro Tipo I.

- **Hipótese Testada (Verdadeira):** $H_0 : \mu = 23$
- **Nível de Significância:** $\alpha = 0.005$

4.2 Código em R

Listing 10: Simulacao de Erro Tipo I em R

```r
# 1. Gerar populacao onde $H_0: \mu=23$ e verdadeira
populacao = rnorm(100000, 23, 17)
mu_h0 = 23
alpha = 0.005
num_simulacoes = 30000

# 2. Inicializar contador
contador = 0

# 3. Loop de Simulacao
for(i in 1:num_simulacoes) {
  amostra = sample(populacao, 20)

  # Realiza o teste t
  p = t.test(amostra,
             alternative = "two.sided",
             mu = mu_h0,
             paired = FALSE,
             conf.level = 1 - alpha)

  # 4. Verifica se houve rejeicao de H0 (Erro Tipo I)
  # Se p-valor < alpha, rejeitamos H0
  contador = ifelse(p$p.value < alpha, 1, 0) + contador
}

# 5. Proporcao  de Erro Tipo I
proporcao_erro = contador / num_simulacoes
print(proporcao_erro)
```

4.3 Código em Python

Listing 11: Simulação de Erro Tipo I em Python

```python
from scipy.stats import ttest_1samp

# 1. Parametros e Populacao (H0: mu=23 e verdadeira)
POP_MEAN = 23
POP_STD = 17
POP_SIZE = 100000
SAMPLE_SIZE = 20
ALPHA = 0.005
NUM_SIMULATIONS = 30000

np.random.seed(42)
population = np.random.normal(POP_MEAN, POP_STD, POP_SIZE)
rejection_count = 0

# 2. Loop de Simulacao
for i in range(NUM_SIMULATIONS):
    sample = np.random.choice(population, SAMPLE_SIZE, replace=False)

    # Usa ttest_1samp para teste t de uma amostra
    t_stat, p_value = ttest_1samp(sample, POP_MEAN)

    # P-valor bicaudal (ttest_1samp ja retorna bicaudal)

```

```python
    # 3. Contagem do Erro Tipo I
    if p_value < ALPHA:
        rejection_count += 1

# 4. Proporcao de Erro Tipo I
proportion_type_i_error = rejection_count / NUM_SIMULATIONS

print(f"Rejeicoes (Erro Tipo I) Observadas: {rejection_count}")
print(f"Proporcao de Erro Tipo I: {proportion_type_i_error:.5f}")
# Espera-se um valor proximo de 0.005
```

5 Simulação do Erro por Múltiplos Testes (Inflação de α)

5.1 Teoria

Ao realizar k testes de hipóteses independentes, a chance de cometer **pelo menos um** Erro Tipo I na "família"de testes (**FWER** - Family-Wise Error Rate) aumenta: $FWER = 1 - (1 - \alpha)^k$.

- **Hipótese Geral (Verdadeira):** $H_0 : \mu_A = \mu_B = \mu_C$
- **Testes Individuais:** $k = 3$ (A vs B, A vs C, B vs C)
- **FWER Teórica:** $1 - (1 - 0.005)^3 \approx 0.0149$

5.2 Código em R

Listing 12: Simulacao de Multiplos Testes em R

```r
# 1. Gerar 3 populacoes com a mesma media (H0 verdadeira)
populacao1 = rnorm(100000, 23, 15)
populacao2 = rnorm(100000, 23, 11)
populacao3 = rnorm(100000, 23, 19)
alpha = 0.005
num_simulacoes = 30000

# 2. Inicializar contador
contador = 0

# 3. Loop de Simulacao
for(i in 1:num_simulacoes) {
  amostra1 = sample(populacao1, 20)
  amostra2 = sample(populacao2, 20)
  amostra3 = sample(populacao3, 20)

  # Realiza os 3 testes t nao pareados (variancias desiguais)
  p1 = t.test(amostra1, amostra2, var.equal = FALSE, conf.level = 1 - alpha)$p.value
  p2 = t.test(amostra1, amostra3, var.equal = FALSE, conf.level = 1 - alpha)$p.value
  p3 = t.test(amostra2, amostra3, var.equal = FALSE, conf.level = 1 - alpha)$p.value

  # 4. Contagem do Erro (se pelo menos 1 teste rejeitou H0)
  contador = ifelse(p1 < alpha | p2 < alpha | p3 < alpha, 1, 0) + contador
}

# 5. Proporcao de Erro (FWER)
proporcao_erro = contador / num_simulacoes
print(proporcao_erro)
```

5.3 Código em Python

Listing 13: Simulação de Múltiplos Testes em Python

```python
from scipy.stats import ttest_ind

# 1. Gerar 3 populacoes com a mesma media (H0 verdadeira)
COMMON_MEAN = 23
STD1, STD2, STD3 = 15, 11, 19
POP_SIZE = 100000
SAMPLE_SIZE = 20
ALPHA = 0.005
NUM_SIMULATIONS = 30000

np.random.seed(42)
pop1 = np.random.normal(COMMON_MEAN, STD1, POP_SIZE)
pop2 = np.random.normal(COMMON_MEAN, STD2, POP_SIZE)
pop3 = np.random.normal(COMMON_MEAN, STD3, POP_SIZE)

rejection_count = 0
NUM_TESTS = 3

# 2. Loop de Simulacao
for i in range(NUM_SIMULATIONS):
    sample1 = np.random.choice(pop1, SAMPLE_SIZE, replace=False)
    sample2 = np.random.choice(pop2, SAMPLE_SIZE, replace=False)
    sample3 = np.random.choice(pop3, SAMPLE_SIZE, replace=False)

    # Realiza os 3 testes t nao pareados (variancias desiguais: equal_var=False)
    _, p_value_1 = ttest_ind(sample1, sample2, equal_var=False)
    _, p_value_2 = ttest_ind(sample1, sample3, equal_var=False)
    _, p_value_3 = ttest_ind(sample2, sample3, equal_var=False)

    # 3. Contagem do Erro por Familia (FWER)
    # Se p1 < alpha OU p2 < alpha OU p3 < alpha
    if (p_value_1 < ALPHA) or (p_value_2 < ALPHA) or (p_value_3 < ALPHA):
        rejection_count += 1

# 4. Proporcao de Erro (FWER)
proportion_error = rejection_count / NUM_SIMULATIONS
fwer_theoretical = 1 - (1 - ALPHA)**NUM_TESTS

print(f"Rejeicoes (FWER) Observadas: {rejection_count}")
print(f"Proporcao de Erro Observada: {proportion_error:.5f}")
print(f"FWER TeOrica: {fwer_theoretical:.5f}")
```